

Docker Compose Guide

A comprehensive guide to using Docker Compose for container orchestration, with commands, examples, and best practices.

By Ravi Kiran Maroju

 **by Ravi Kiran**



Getting Started with Docker Compose

By Ravi Kiran Maroju

Save this in a file

Create a file called docker-compose.yml in a folder.

Start everything

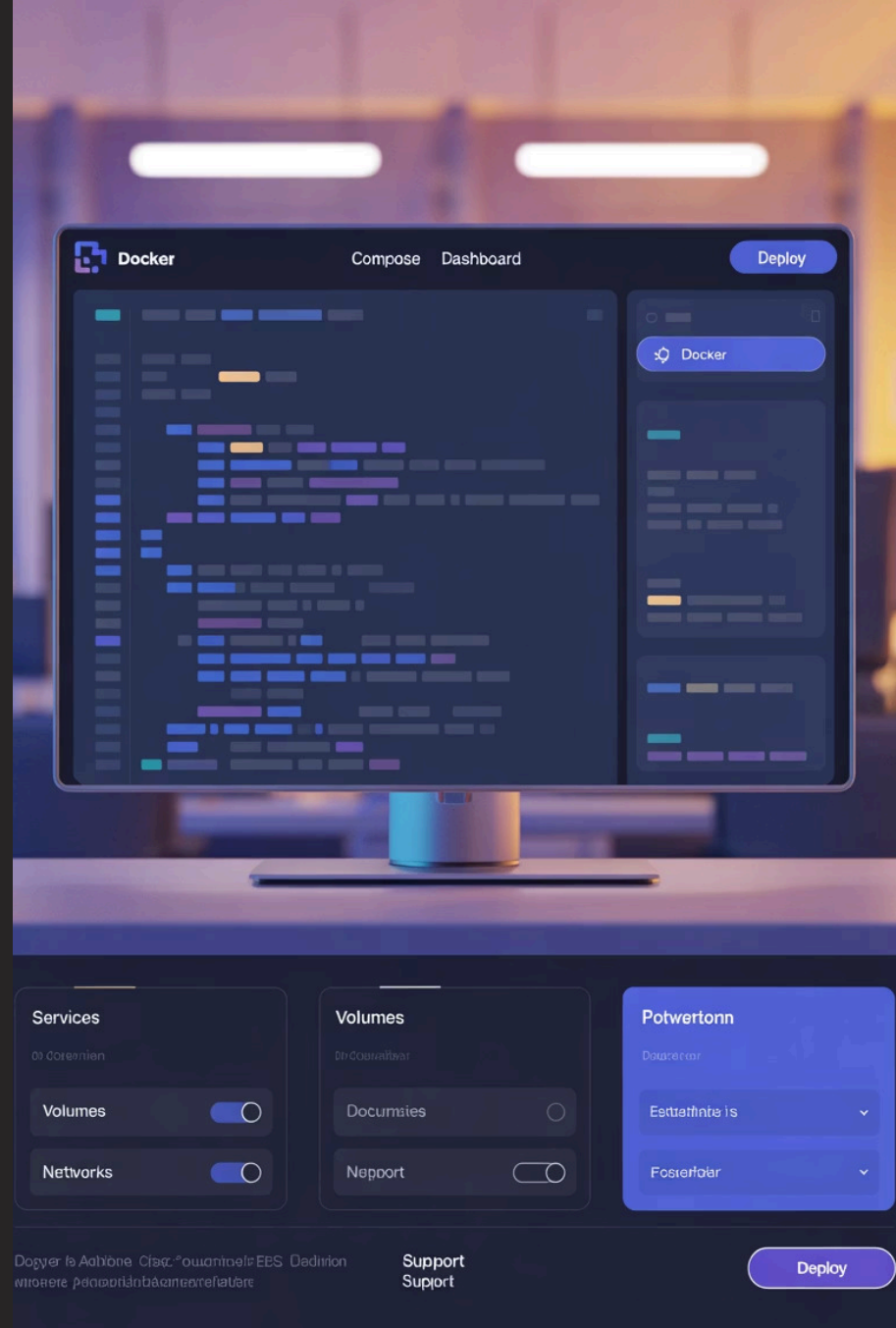
`docker-compose up` → This builds and runs all the containers!

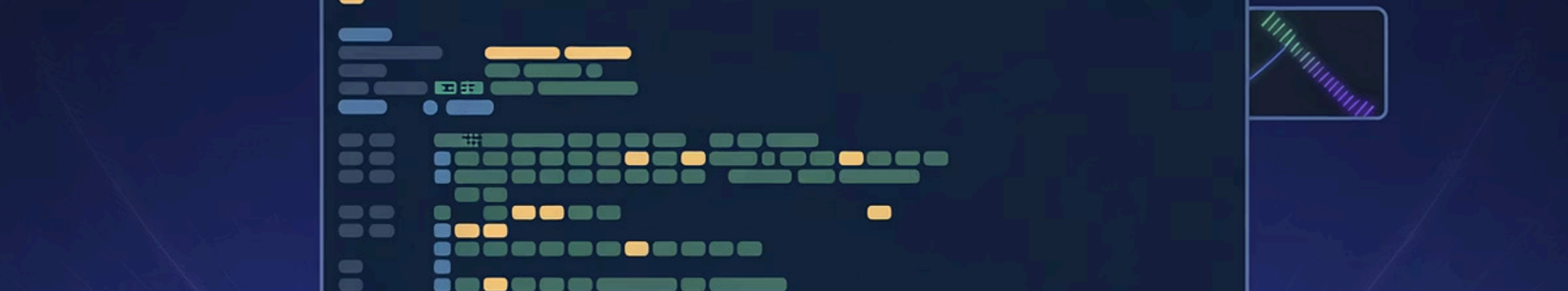
Run in background

`docker-compose up -d`

Stop everything

`docker-compose down` → This stops and removes containers, networks, and volumes created by up.





Other Compose Commands

By Ravi Kiran Maroju



docker-compose build

Builds images from a Dockerfile inside Compose 🛠️



docker-compose logs

Shows what each container is printing in real time 📺



docker-compose exec

Go inside a running container (like a secret tunnel) 🚶



docker-compose ps

Shows which containers are running 🚦

Scaling Containers

By Ravi Kiran Maroju



docker-compose scale web=3

Run 3 copies of the web container 🚀🚀🚀



Important Note

Scale works only if containers can run multiple copies (e.g., web servers).





Using Environment Variables

By Ravi Kiran Maroju

What are they?

They are like little secret notes



you pass to a container:

"Hey container, your password is mypassword"

You can use them inside docker-compose.yml.

Example in docker-compose.yml

environment:

```
MYSQL_ROOT_PASSWORD:
${DB_PASSWORD}
```

Create a .env file

Then create a .env file in the same folder:

```
DB_PASSWORD=mypassword
```



Now your password is not written directly in the YAML file!



Docker Compose Explained Simply

By Ravi Kiran Maroju

Term	Like This...
docker-compose.yml	Recipe for building LEGO sets 🧱
docker-compose up	Start all your LEGOs at once 🚦
docker-compose down	Pack away your LEGOs 🧺
build	Create your LEGO pieces 📦
logs	See what your toys are saying 📢
exec	Go inside the toy 🚪
.env file	Secret note for your toys 📧



Mini Project Idea

By Ravi Kiran Maroju



Create a folder

Make a new directory for your project



Create docker-compose.yml

```
version: "3"
```

```
services:
```

```
hello:
```

```
image: busybox
```

```
command: sh -c "while true; do echo Hello from container; sleep 2; done"
```



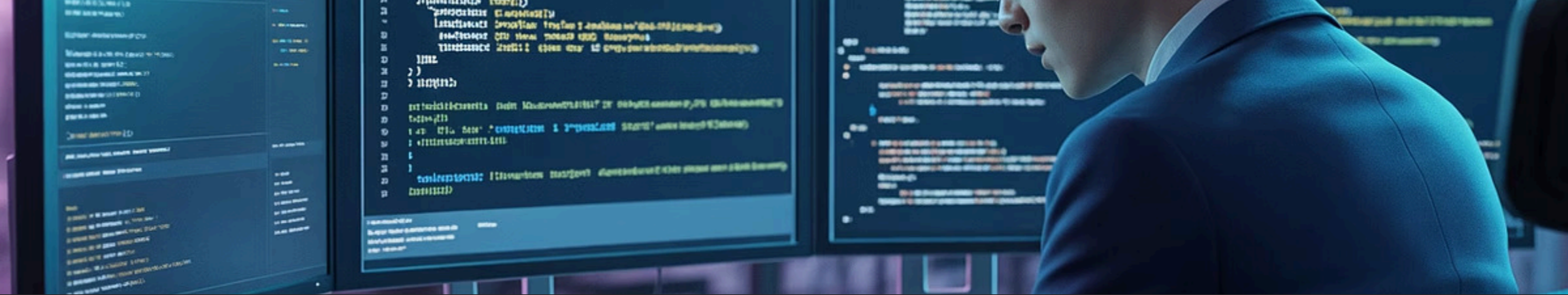
Run the command

```
docker-compose up
```



See the result

✅ It will print "Hello from container" every 2 seconds.



Common Docker Compose Commands

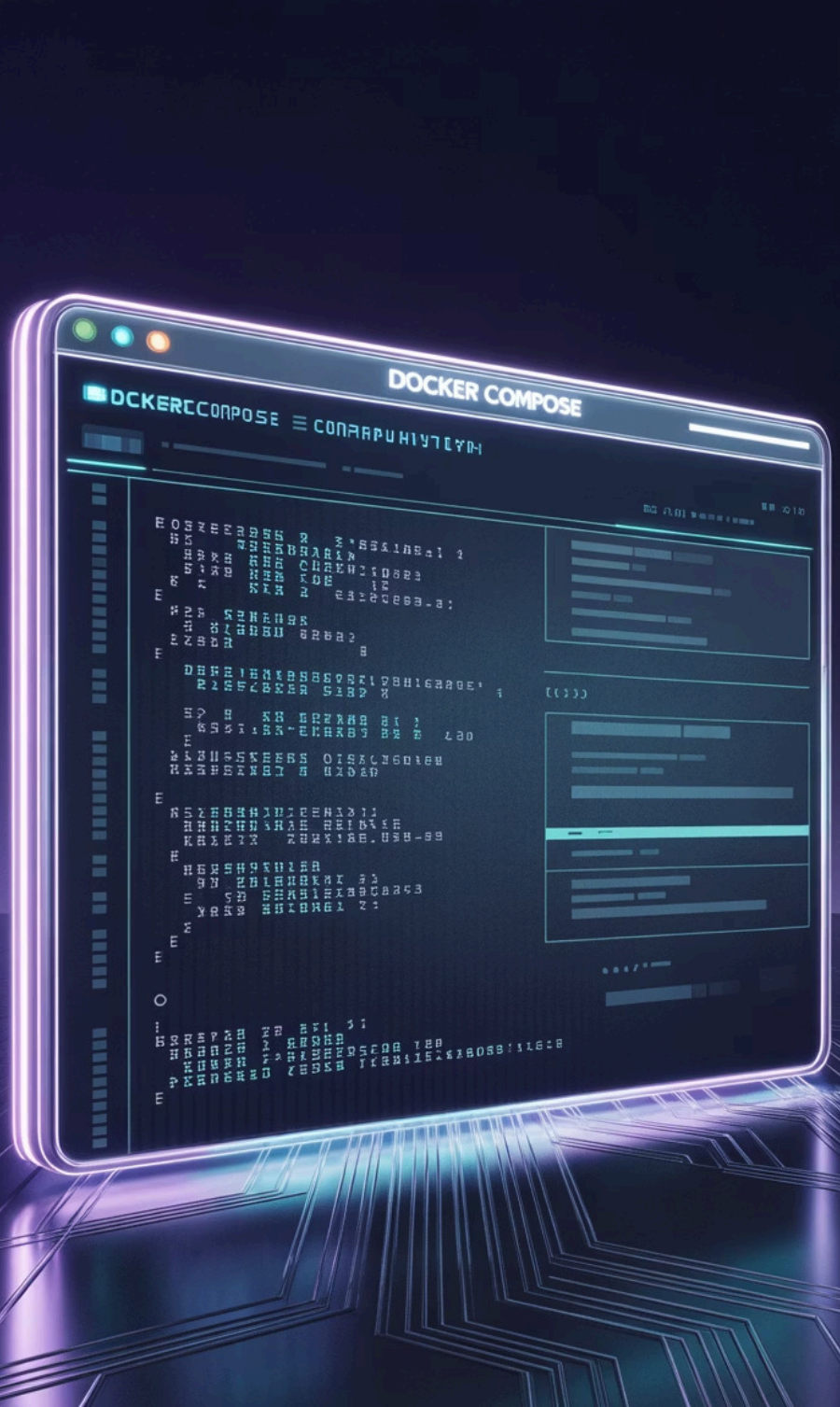
By Ravi Kiran Maroju

Command	What it does	Example
<code>docker-compose up</code>	Starts all the services (containers) in the <code>docker-compose.yml</code> file.	<code>docker-compose up</code>
<code>docker-compose up -d</code>	Starts services in the background ("detached mode").	<code>docker-compose up -d</code>
<code>docker-compose down</code>	Stops and removes everything started by up (containers, networks, etc.).	<code>docker-compose down</code>
<code>docker-compose stop</code>	Stops containers only (does not remove them).	<code>docker-compose stop</code>

More Docker Compose Commands

By Ravi Kiran Maraju

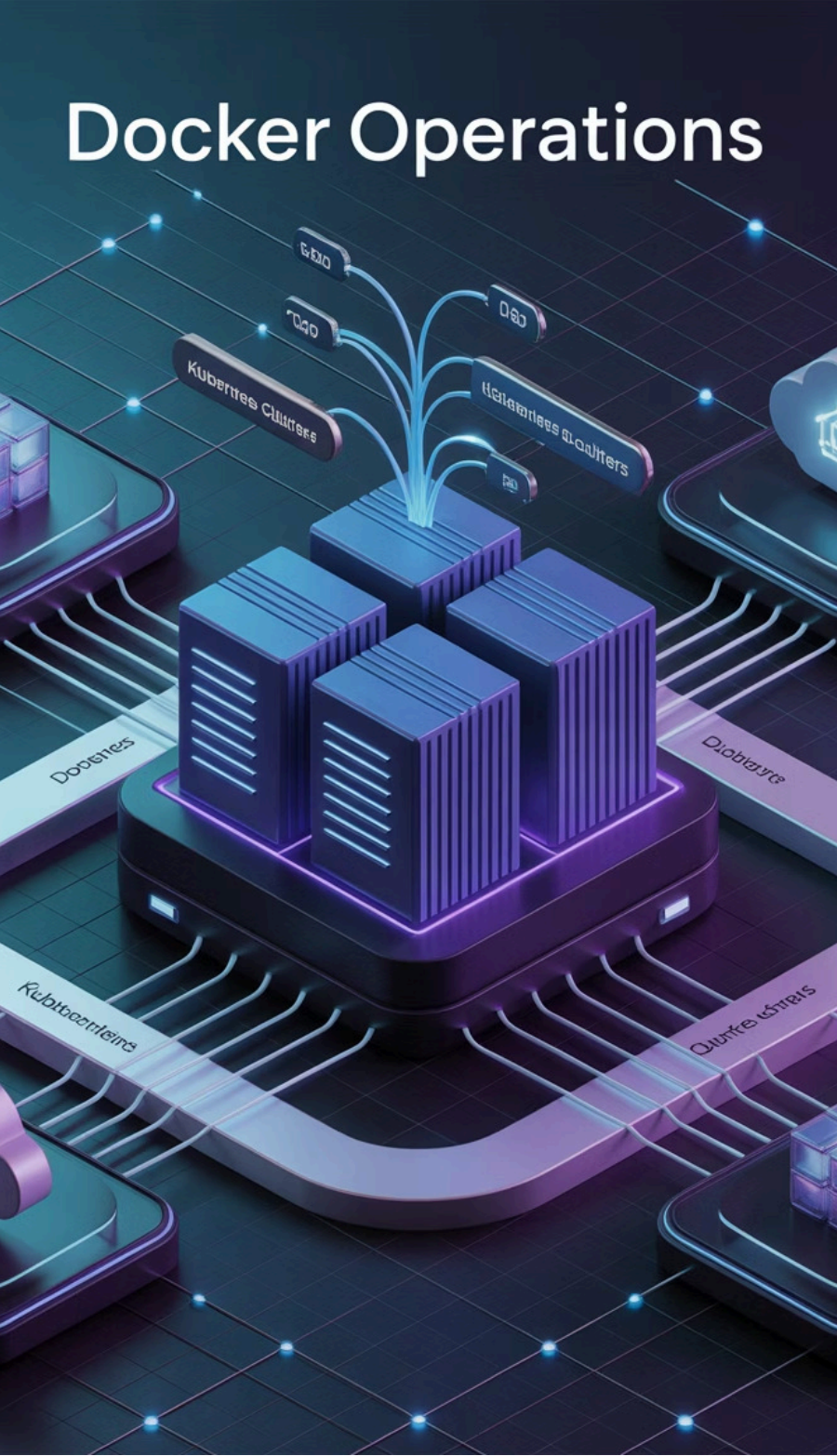
Command	What it does	Example
<code>docker-compose start</code>	Starts previously stopped containers.	<code>docker-compose start</code>
<code>docker-compose restart</code>	Restarts running containers.	<code>docker-compose restart</code>
<code>docker-compose build</code>	Builds the images from Dockerfiles defined in the services.	<code>docker-compose build</code>
<code>docker-compose pull</code>	Pulls required images from Docker Hub.	<code>docker-compose pull</code>



Advanced Docker Compose Commands

By Ravi Kiran Maroju

Command	What it does	Example
<code>docker-compose push</code>	Pushes built images to a registry (if tags and registry are defined).	<code>docker-compose push</code>
<code>docker-compose logs</code>	Shows log output (like a diary) of the containers.	<code>docker-compose logs</code>
<code>docker-compose logs -f</code>	Shows logs live (like a TV broadcast).	<code>docker-compose logs -f</code>
<code>docker-compose exec</code>	Run a command inside a running container.	<code>docker-compose exec web bash</code>





Container Management Commands

By Ravi Kiran Maroju

Command	What it does	Example
<code>docker-compose run</code>	Starts a new container and runs a command.	<code>docker-compose run web ls /</code>
<code>docker-compose ps</code>	Lists all running containers in the Compose project.	<code>docker-compose ps</code>
<code>docker-compose config</code>	Validates and shows the final merged Compose config.	<code>docker-compose config</code>
<code>docker-compose rm</code>	Removes stopped containers.	<code>docker-compose rm</code>

Dockerflow

Utility Docker Compose Commands

By Ravi Kiran Maroju

Command	What it does	Example
<code>docker-compose top</code>	Shows the processes running inside the containers.	<code>docker-compose top</code>
<code>docker-compose kill</code>	Forcefully stops containers (like "stop now!").	<code>docker-compose kill</code>
<code>docker-compose scale</code>	Runs multiple copies (replicas) of a service.	<code>docker-compose scale web=3</code> (only in v2)
<code>docker-compose version</code>	Shows the Docker Compose version.	<code>docker-compose version</code>
<code>docker-compose help</code>	Shows help for Docker Compose commands.	<code>docker-compose help</code>

Docker Compose YAML File Structure

By Ravi Kiran Maroju



Important docker-compose.yml Options

These are the keywords used inside the YAML file

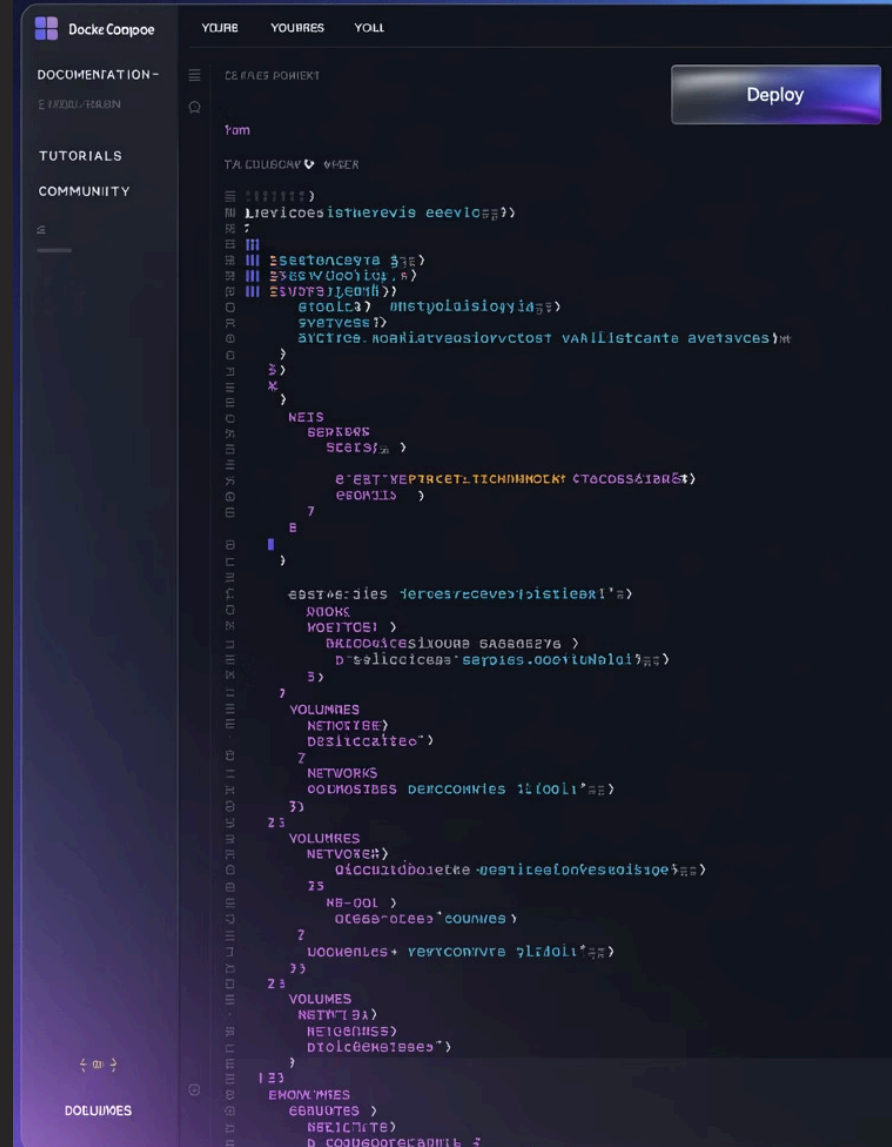


Hierarchical Format

YAML uses indentation to define the hierarchy of configuration options

Basic Structure

The file contains services, networks, volumes, and other configuration options



Version Definition

By Ravi Kiran Maroju

version

Defines the Compose file version (e.g., "3").

Example: version: "3"

Why Version Matters

Different versions support different features and syntax

Version 3 is recommended for most modern Docker installations

Compatibility

Newer versions may not work with older Docker installations

Always check compatibility between Docker Engine and Compose versions



Services Configuration

By Ravi Kiran Maroju

services

Lists the containers to be started.

Example: services: web:

Service Definition

Each service represents a container that will be created

Services can be configured with various options

Service Names

Choose meaningful names for your services

Service names are used as hostnames in the Docker network

Containerized Architecture: Orchestrating Modern Applications”



Image Configuration

By Ravi Kiran Maroju

image

The image name from Docker Hub or built locally.

Example: image: nginx

Image Tags

You can specify a specific version using tags

Example: image: nginx:1.19

Private Registries

You can use images from private registries

Example: image: registry.example.com/myapp:latest



Pull images

Monitor your
container ecosystem
in real-time

Build Configuration

By Ravi Kiran Maroju

build

Tells Compose to build the image from a Dockerfile.

Example: build: .

Build Context

Specify the directory containing the Dockerfile

Example: build: ./app

Advanced Build Options

You can specify Dockerfile path, build arguments, and more

Example: build: { context: ./app, dockerfile: Dockerfile.dev }





Container Naming

By Ravi Kiran Maroju

container_name

Gives a specific name to your container.

Example: container_name:
my_app

Why Name Containers?

Makes it easier to identify containers in logs and commands

Provides a consistent reference for external tools

Naming Limitations

Container names must be unique across your Docker host

Using container_name prevents scaling the service



Port Mapping

By Ravi Kiran Maroju

ports

Maps ports between container and host.

Example: ports: ["8080:80"]

Port Syntax

Format is

"HOST_PORT:CONTAINER_PORT"

You can specify protocol:

"8080:80/tcp"

Multiple Port Mappings

A service can expose multiple ports

Example: ports: ["8080:80",
"8443:443"]



Volume Configuration

By Ravi Kiran Maroju

volumes

Mounts a folder or named volume.

Example: volumes: ["/data:/data"]

Types of Volumes

Host path volumes: "/host-path:/container-path"

Named volumes: "volume-name:/container-path"

Volume Options

You can specify read-only volumes: "/data:/data:ro"

Volumes persist data even when containers are removed



Environment Variables

By Ravi Kiran Maroju

environment

Sets environment variables.

Example: environment:

```
MYSQL_ROOT_PASSWORD=1234
```

Multiple Variables

You can set multiple environment variables

Use array or dictionary syntax in YAML

Variable References

Reference host environment variables with `${VAR_NAME}`

Example: environment: `PASSWORD=${DB_PASSWORD}`

Docker environment files loaded in containers

Edgmonte freunt fiil sopwadeetis erinéd to éorjoleoaimenot Gogeúment Contresl;

Create Environment



4,401.20

Container		
Environment Variables	Environment Variables	Port Mappings
12.1207.886+ 7.8 118.63.0.0.0 C	0.8.2207.889+ 7.8 108.63.0.0.0 C	18.1207.889+ 7.8 108.63.0.0.0 C
Enccolonecon Gólinnert	Enccolonecon Gólinnert	Enccolonecon Gólinnert
Port Mappings	Port Mappings	Port Mappings
20.11.0.0.0 100.0.0.0.0 C	20.11.0.0.0 100.0.0.0.0 C	20.11.0.0.0 100.0.0.0.0 C

Environment Files

By Ravi Kiran Maraju

env_file

Loads variables from a .env file.

Example: env_file: .env

Multiple Env Files

You can specify multiple environment files

Example: env_file: [".env.dev", ".env.secrets"]

Env File Format

Simple KEY=VALUE format, one per line

Comments start with #



Command Configuration

By Ravi Kiran Maraju

command

Overrides the default command in the Docker image.

Example: command: npm start

Command Formats

String format: command: "npm start"

Array format: command: ["npm", "start"]

Use Cases

Override the default behavior of an image

Pass additional arguments to the default command

DOCKER SERVICES WITH DEPENDENCY RELATIONSHIPS

Dependency Management

By Ravi Kiran Maroju

depends_on

Defines the startup order of containers.

Example: `depends_on: [db]`

Startup Order

Services start in dependency order

Dependent services wait for dependencies to start

Limitations

Only waits for container to start, not for service to be ready

For advanced health checks, use healthcheck option

Restart Policies

By Ravi Kiran Maroju

restart

Defines restart policy. Options:
no, always, on-failure, unless-stopped

Example: restart: always

Policy Options

no: Never restart (default)

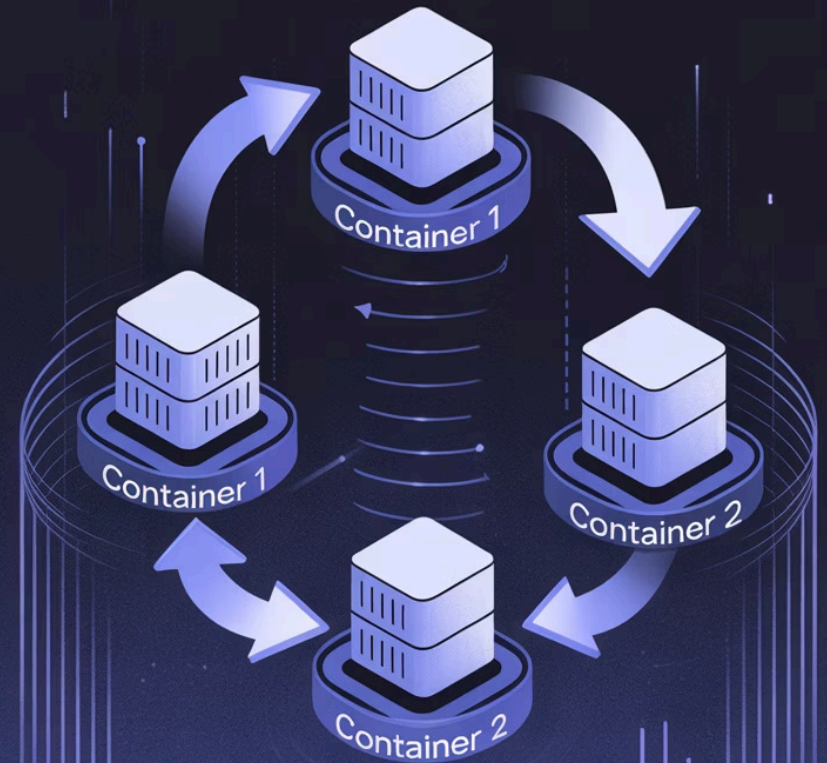
always: Always restart, even
after docker-compose down

More Policy Options

on-failure: Restart only if container exits with error

unless-stopped: Always restart unless explicitly stopped

Docker Restart Policy



Network Configuration

By Ravi Kiran Maroju

networks

Assigns containers to networks.

Example: networks:
[my_network]

Network Types

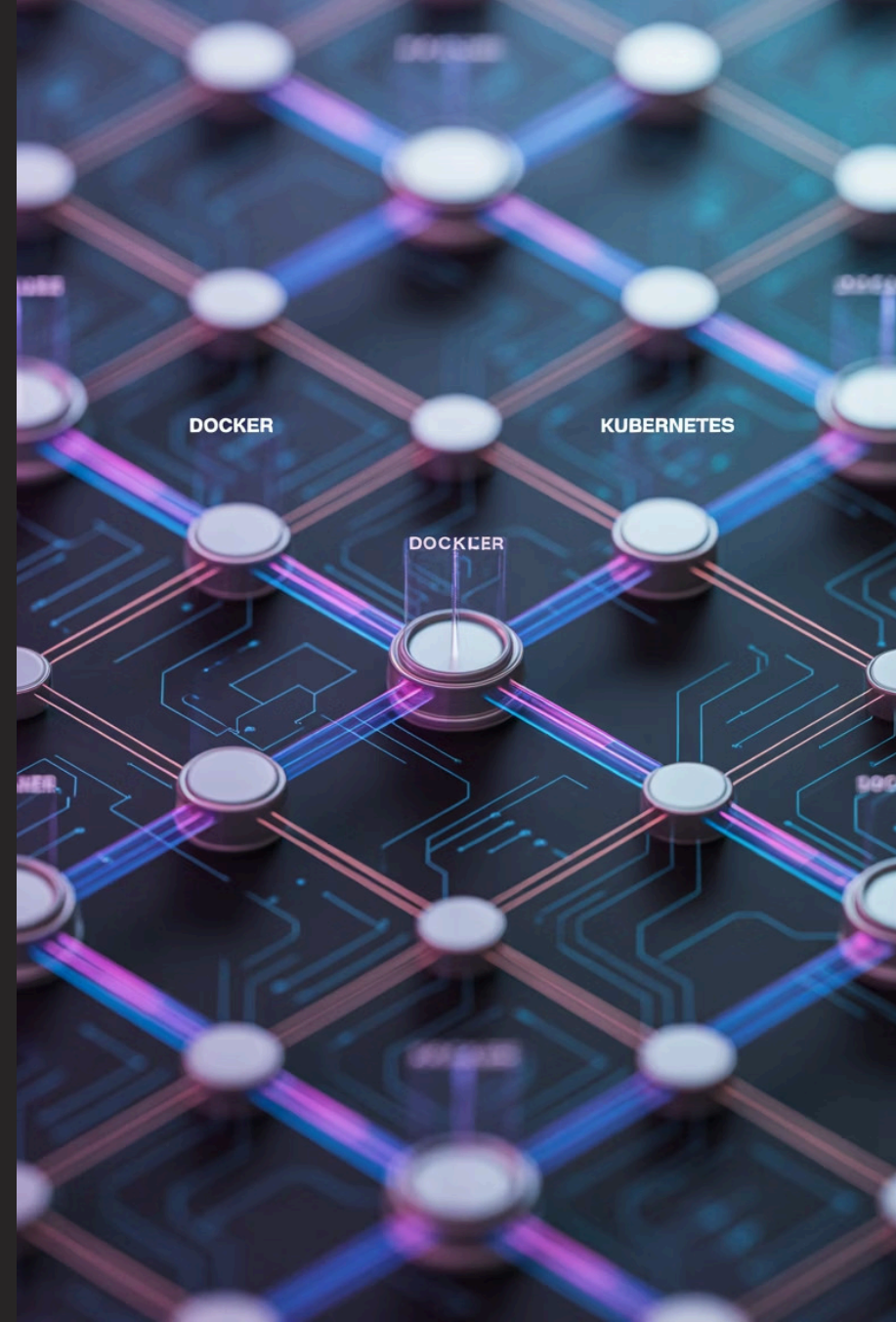
bridge: Default isolated network
on single host

host: Use the host's networking
directly

Custom Networks

Define custom networks with specific drivers

Connect services to multiple networks



Legacy Links

By Ravi Kiran Maroju

links

Connects to another container (legacy, use networks instead).

Example: links: [db]

Deprecated Feature

Links are a legacy feature from earlier Docker versions

Modern Docker Compose uses networks instead

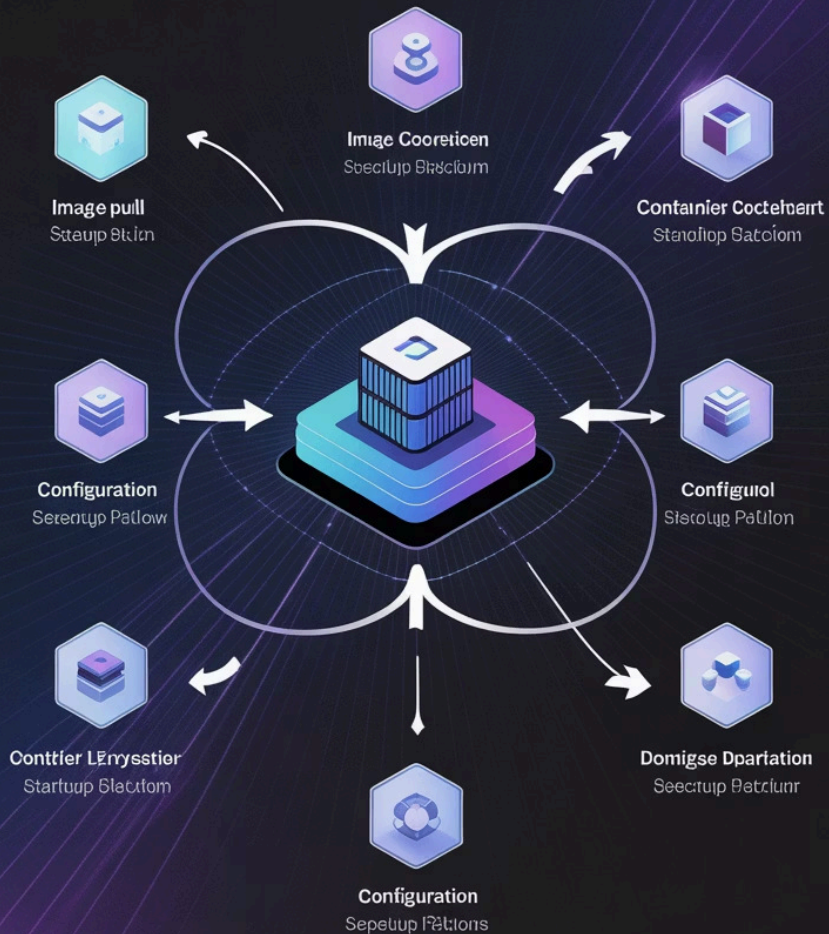
Migration Path

Replace links with proper network configuration

Services on the same network can communicate by service name



Docker Entrypoint Execution Flow



Entrypoint Configuration

By Ravi Kiran Maroju

entrypoint

Overrides the Docker image entrypoint.

Example: entrypoint: ["sh", "-c"]

Entrypoint vs Command

Entrypoint defines the executable that runs

Command provides arguments to the entrypoint

Use Cases

Override the default entrypoint of an image

Add wrapper scripts or initialization logic

Health Checks

By Ravi Kiran Maroju

healthcheck

Checks if a container is healthy.

Example: healthcheck: { test: "curl -f http://localhost", interval: 30s }

Health Check Options

test: Command to run to check health

interval: How often to check

Additional Options

timeout: Maximum time for check to complete

retries: Number of consecutive failures before unhealthy

start_period: Initial grace period

Docker Container Health Monitoring

Healthy initial connection to container, apply latest updates, monitor container health, detect issues





Logging Configuration

By Ravi Kiran Maroju

logging

Controls logging options (driver, etc.).

Example: logging: { driver: "json-file" }

Log Drivers

json-file: Default JSON logging

syslog: Log to syslog

journald: Log to journald

Log Options

max-size: Maximum log file size

max-file: Maximum number of log files

Example: logging: { options: { max-size: "10m", max-file: "3" } }



Secrets Management

By Ravi Kiran Maroju

secrets

For managing secret files securely (Docker Swarm).

Example: secrets: [my_secret]

Secret Sources

File-based secrets: Read from a file on the host

External secrets: Managed by Docker Swarm

Accessing Secrets

Secrets are mounted at `/run/secrets/SECRET_NAME`

Only available in Swarm mode

Configuration Files

By Ravi Kiran Maroju

configs

For managing configuration files
(Swarm only).

Example: configs: [my_config]

Config vs Secrets

Configs are for non-sensitive
configuration data

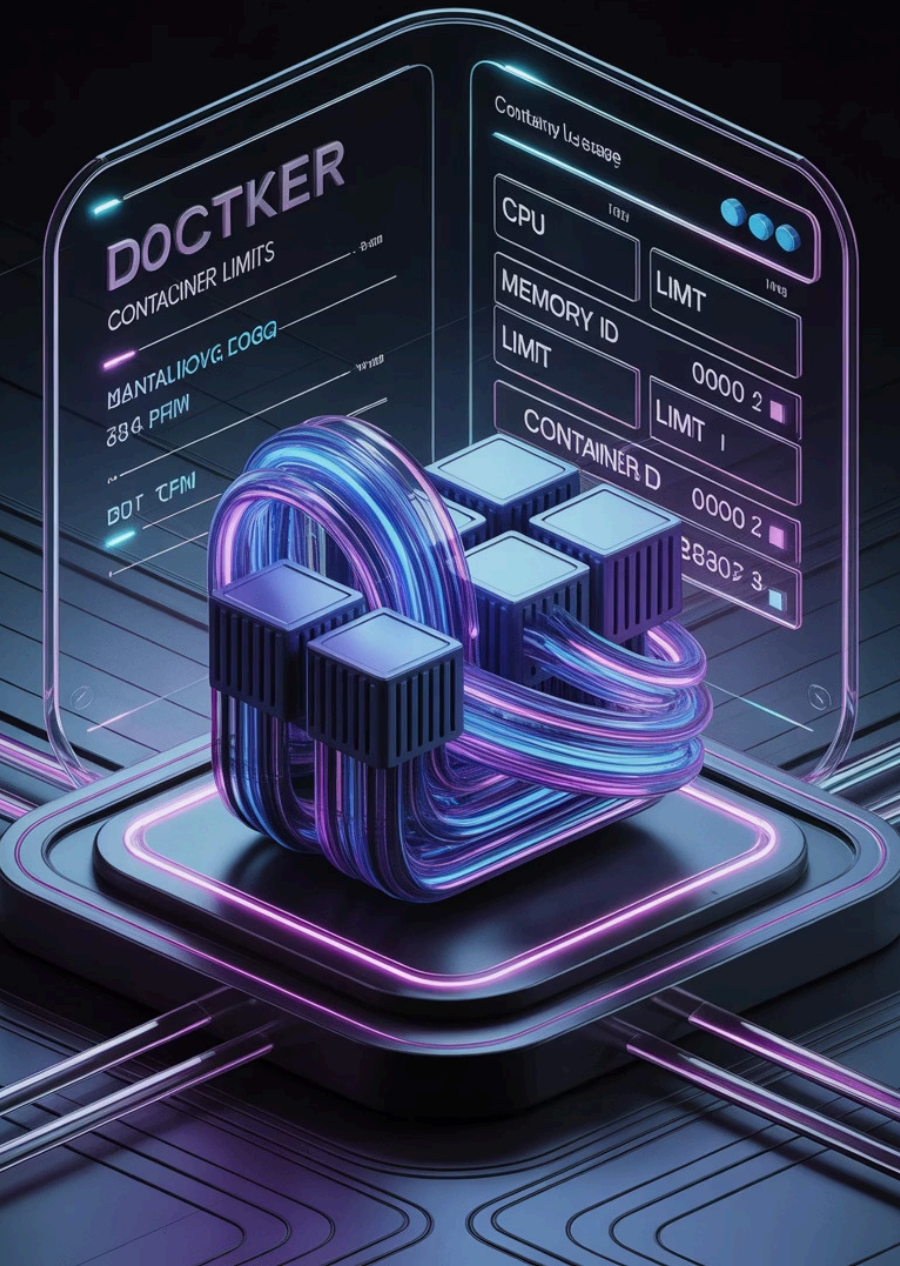
Similar to secrets but not
encrypted

Config Sources

File-based configs: Read from a file on the host

External configs: Managed by Docker Swarm





Resource Limits

By Ravi Kiran Maroju

ulimits

Sets resource limits.

Example: `ulimits: { nproc: 65535 }`

Common Limits

`nproc`: Maximum number of processes

`nofile`: Maximum number of open files

Soft and Hard Limits

Specify both soft and hard limits

Example: `ulimits: { nofile: { soft: 20000, hard: 40000 } }`

Linux Capabilities

By Ravi Kiran Maroju

cap_add / cap_drop

Adds or removes Linux capabilities.

Example: cap_add:
["NET_ADMIN"]

Common Capabilities

NET_ADMIN: Network administration

SYS_PTRACE: Process tracing

SYS_ADMIN: System administration (powerful, use with caution)

Security Considerations

Only add capabilities that are absolutely necessary

Drop capabilities to improve container security





Privileged Mode

By Ravi Kiran Maroju

privileged

Gives full access to the host (not recommended for beginners).

Example: `privileged: true`

Security Implications

Privileged containers have nearly all capabilities of the host

Can access host devices and modify host settings

Alternatives

Use specific capabilities with `cap_add` instead

Mount only required devices with `devices` option

Command Flags: Build Before Starting

By Ravi Kiran Maroju



`docker-compose up --build`

Builds images before starting containers

When to Use

Use when you've made changes to your Dockerfile

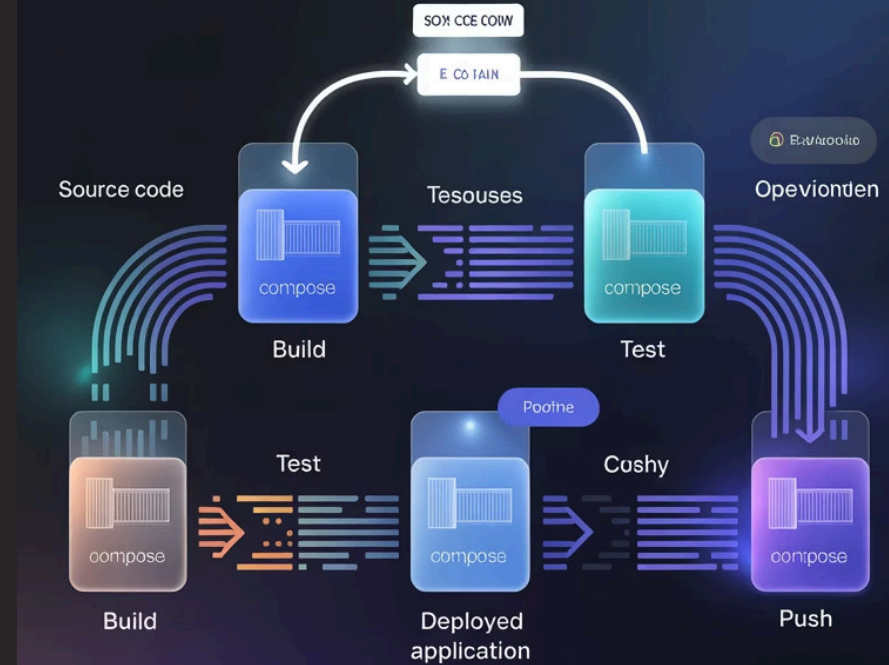
Ensures containers use the latest image version



How It Works

Forces rebuild of images even if they exist

Then starts containers using the newly built images



Automate your container workflows

Outatien contetie your containenb col eahatveirevylital ding. bñlðl-
acrdre rauvovexiod; onettunejpaub anr ldelve yaponte ðuvof the ling the
fnealut thelig luics, eurprciutiw aol a theitt aa lo soctcoieceors.



Command Flags: Remove Volumes

By Ravi Kiran Maroju



**docker-compose down
-v**

Also removes volumes
created



Data Loss Warning

This will delete all data in
named volumes

Use with caution in
production environments



Use Cases

Clean up development environments completely

Reset databases to initial state

Docker Orphaned Containers Being Cleaned Up



Command Flags: Remove Orphans



docker-compose up -- remove-orphans

Removes containers not
defined in the current
docker-compose.yml



What are Orphans?

Containers from previous
Compose runs that are no
longer defined

Often occur when you
remove services from your
compose file



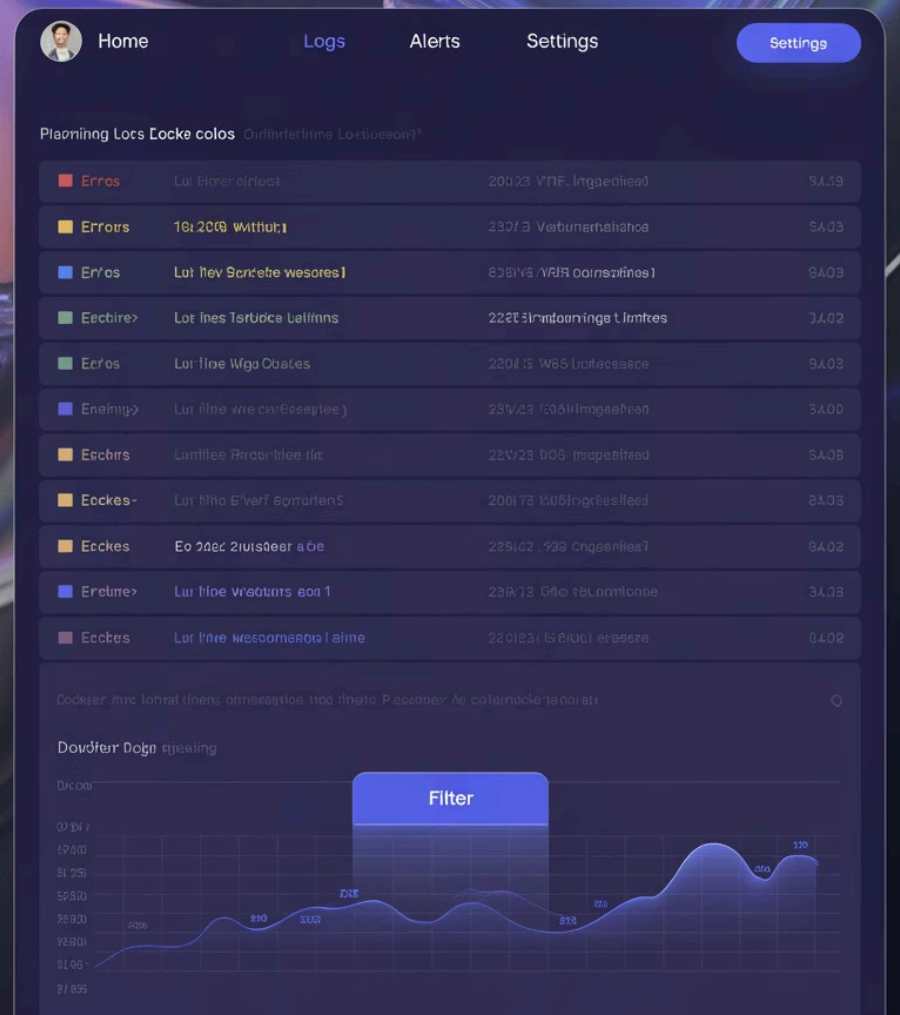
Benefits

Keeps your Docker environment clean

Prevents resource usage by unused containers

By Ravi Kiran Maroju

Docker



Command Flags: Limit Log Output

By Ravi Kiran Maroju



docker-compose logs -tail=50

Shows the last 50 lines of logs



Filtering Options

Limit output to specific services: docker-compose logs web

Combine with follow: docker-compose logs --tail=50 -f



Time-Based Filtering

Filter by time: docker-compose logs --since 2020-01-01

Relative time: docker-compose logs --since 10m

Command Flags: Non-Interactive Mode

By Ravi Kiran Maroju



docker-compose exec -T

Disable pseudo-TTY (for scripts)



Automation Use Case

Perfect for running commands from CI/CD pipelines

Allows output to be captured by other programs



Example

```
docker-compose exec -T web pytest > test-results.txt
```

Runs tests and captures output to a file

Docker

Dashboard

Containers

Networks

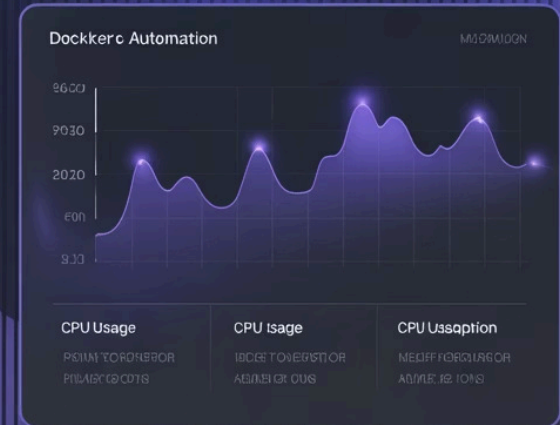
Logs

Deploy

Orchestrate your workflow

Deploy →

Conosunption



Command Flags: Auto-Remove Containers

By Ravi Kiran Maroju



docker-compose run -rm

Automatically remove container after it exits



Cleanup Benefits

Prevents accumulation of stopped containers

Keeps your Docker environment clean



Common Use Cases

Running one-off tasks like database migrations

Executing tests that don't need to persist



Environment Variables in .env File

By Ravi Kiran Maroju



Secret Keeper

You can create a .env file to store variables



.env File Format

```
DB_PASSWORD=mypassword
```

```
APP_PORT=3000
```



Usage in docker-compose.yml

environment:

```
- MYSQL_ROOT_PASSWORD=${DB_PASSWORD}
```



Environment Variables Best Practices

By Ravi Kiran Maroju



Add .env to .gitignore

Never commit sensitive environment variables to version control



Provide a template

Include a .env.example file with dummy values as a template

3

Validate required variables

Add checks in your application to verify required variables exist



Use secrets for sensitive data

For production, consider Docker secrets instead of environment variables

Docker Environment Variables Security Best Practices



Docker Compose Command Summary

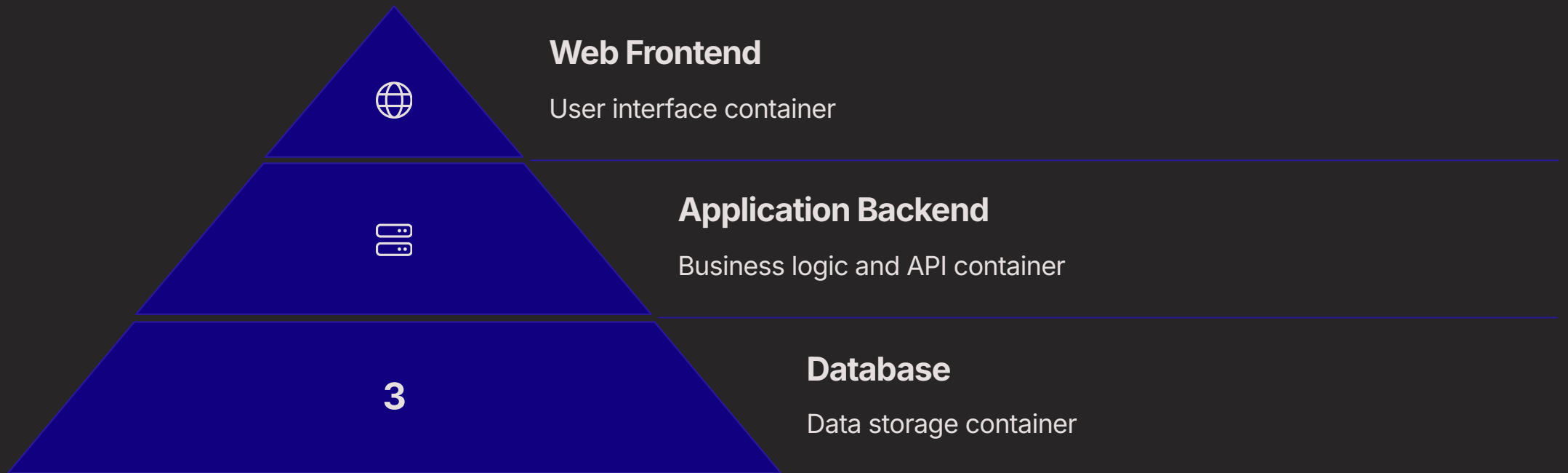
By Ravi Kiran Maroju

What You Want To Do	Use This
Start services	docker-compose up
Stop services	docker-compose down
Build from Dockerfile	docker-compose build
Logs	docker-compose logs
Enter container	docker-compose exec web bash
Use .env secrets	Create .env + use \${VAR}
Run 3 of same service	docker-compose scale web=3



Real World Example: Web Application

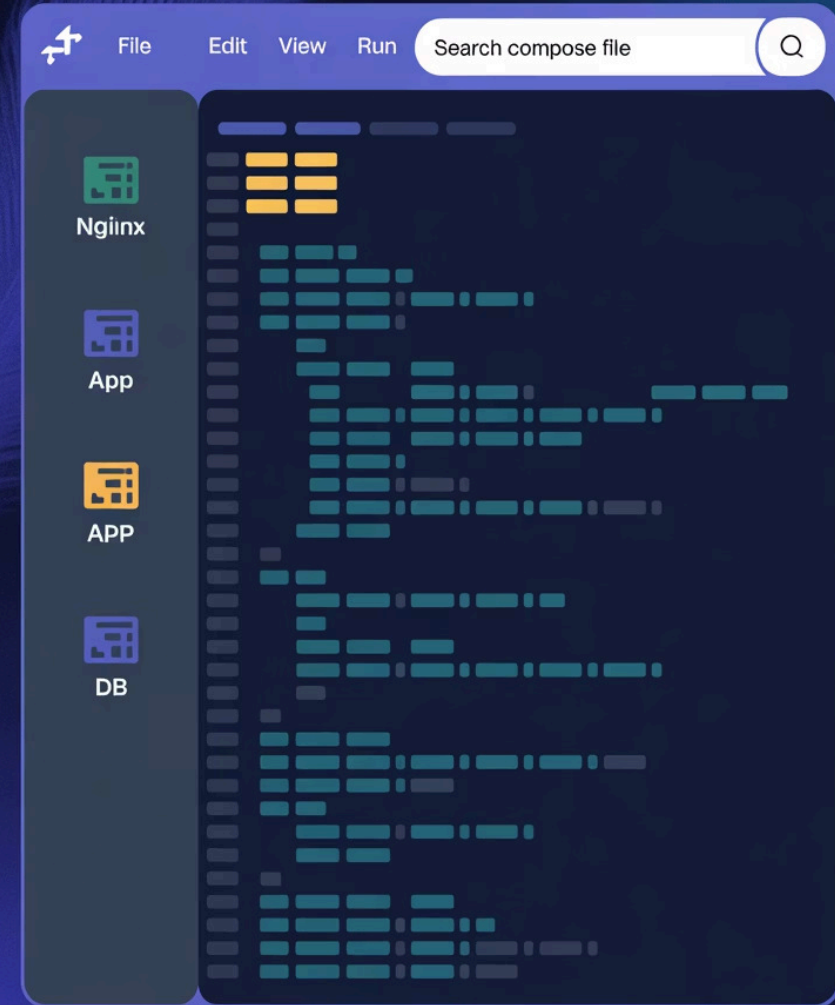
By Ravi Kiran Maroju



Real World Example: Configuration

By Ravi Kiran Maroju

```
version: '3'
services:
  app:
    build: .
    ports:
      - "5000:5000"
    environment:
      - DEBUG=true
  redis:
    image: redis
```

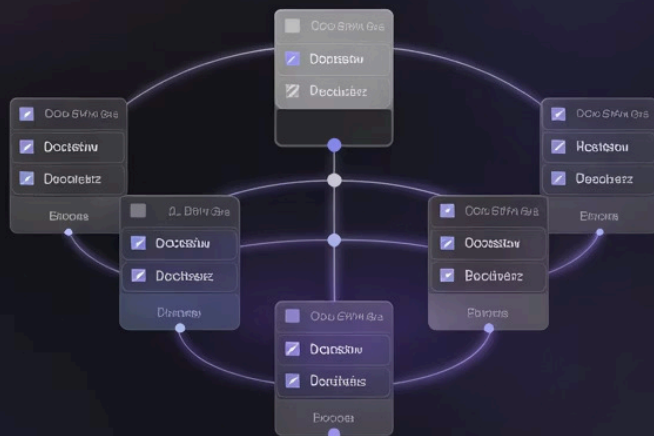


By Ravi Kiran Maroju

What This Does

- Builds and starts a Python app
- Starts Redis from Docker Hub
- Maps ports and adds environment variables

Compose Insights



Extending Docker Compose Files

By Ravi Kiran Maroju



Base Configuration

docker-compose.yml with common settings



Environment-Specific Overrides

docker-compose.override.yml for development

docker-compose.prod.yml for production



Running with Overrides

docker-compose -f docker-compose.yml -f docker-compose.prod.yml up

Docker Compose Networking

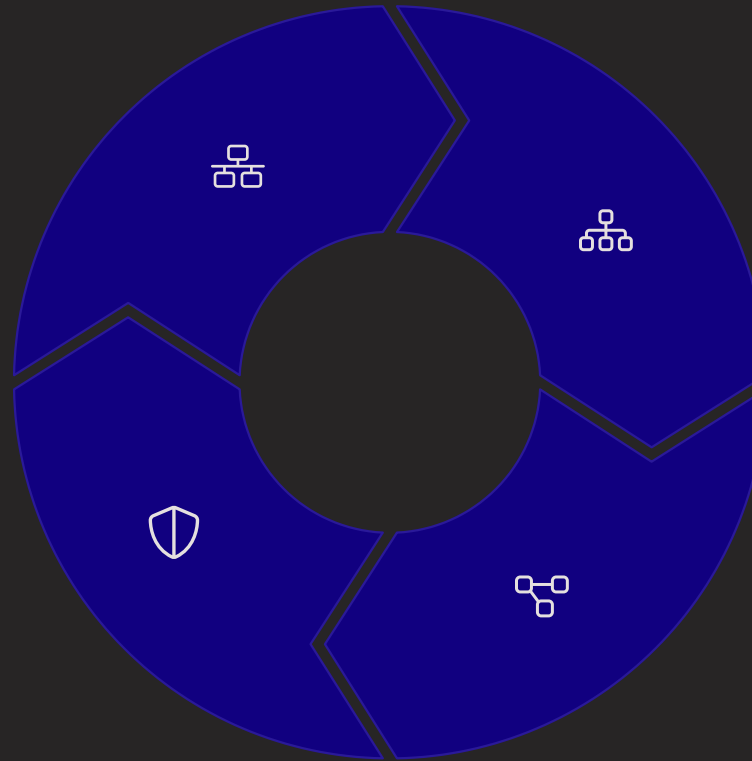
By Ravi Kiran Maroju

Default Network

All services join a default network

Network Security

Control which services can communicate



Service Discovery

Services can reach each other by name

Custom Networks

Create isolated network segments

Docker Compose Volumes

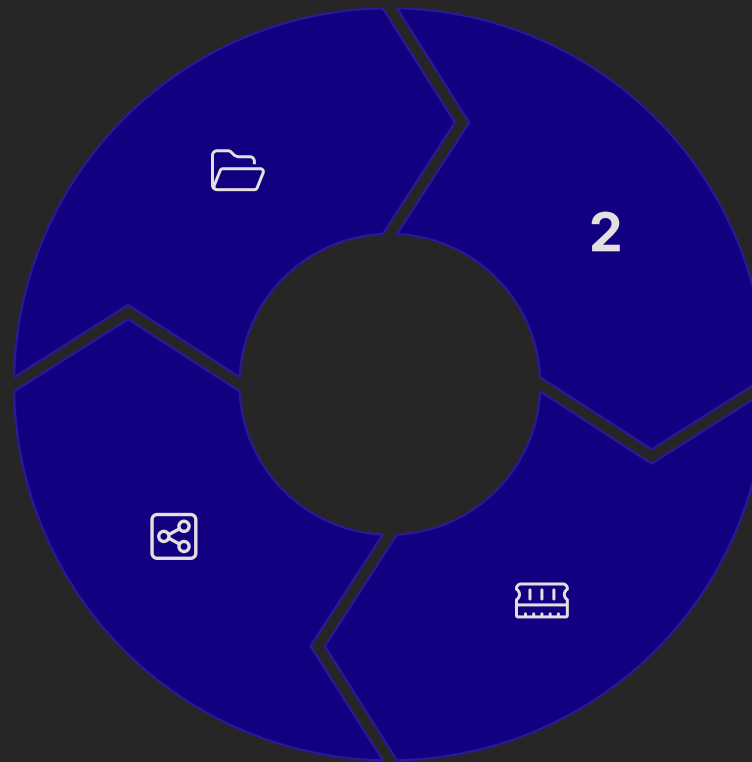
By Ravi Kiran Maroju

Host Volumes

Map host directories to container paths

Shared Volumes

Multiple containers accessing the same data



Named Volumes

Persistent storage managed by Docker

Anonymous Volumes

Temporary volumes tied to container lifecycle

Docker Compose for Development

By Ravi Kiran Maroju

Code Hot Reloading

Mount source code as a volume

Changes reflect immediately
without rebuilding

Debug Mode

Enable debug options in
development

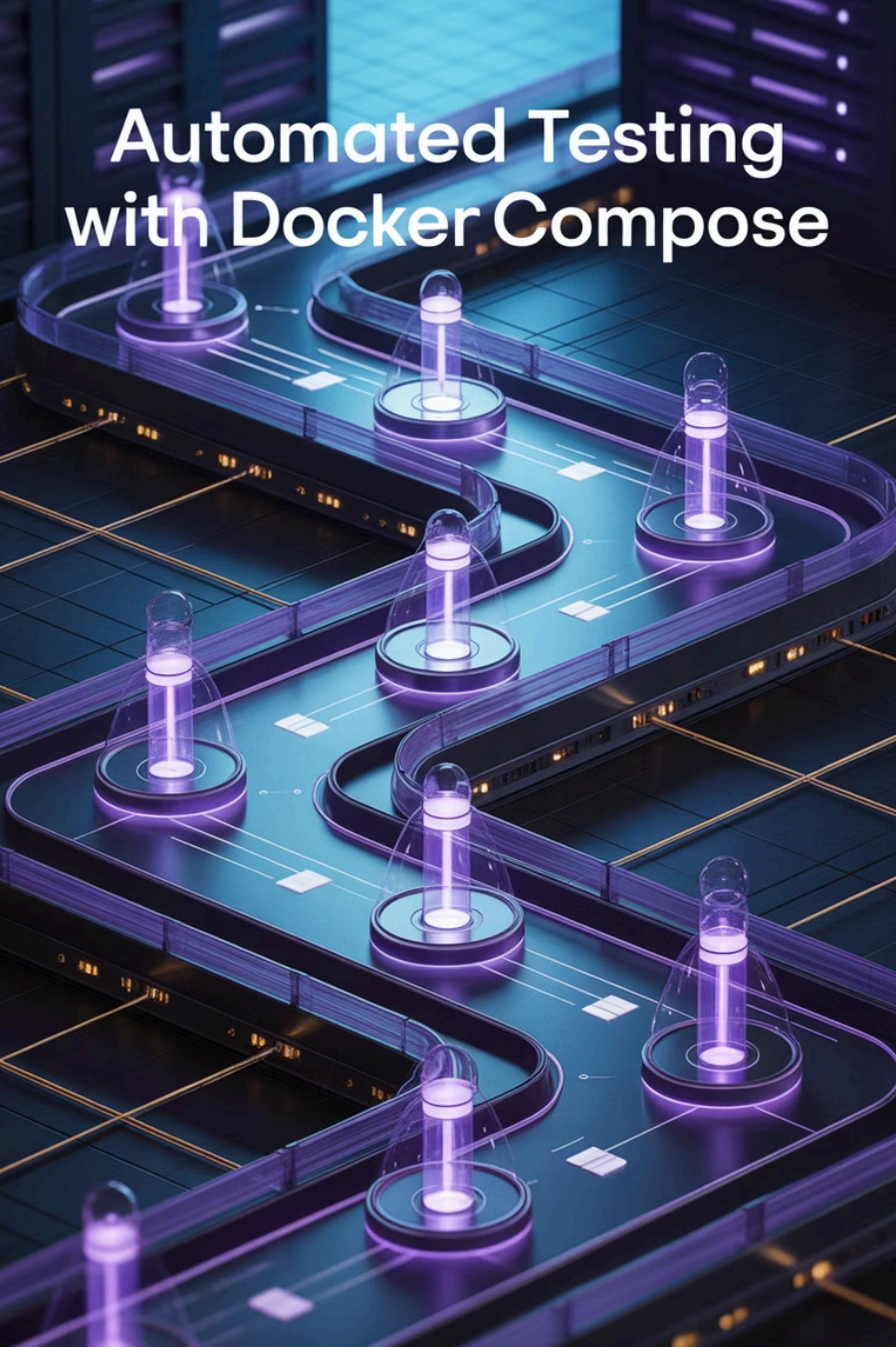
Expose debug ports for IDE
integration

Local Dependencies

Run databases and services locally

Avoid dependency on external services





Automated Testing with Docker Compose

Docker Compose for Testing

By Ravi Kiran Maroju

Integration Tests

Spin up complete environments
for testing

Test interactions between
services

Isolated Test Runs

Create fresh environments for
each test run

Avoid test interference

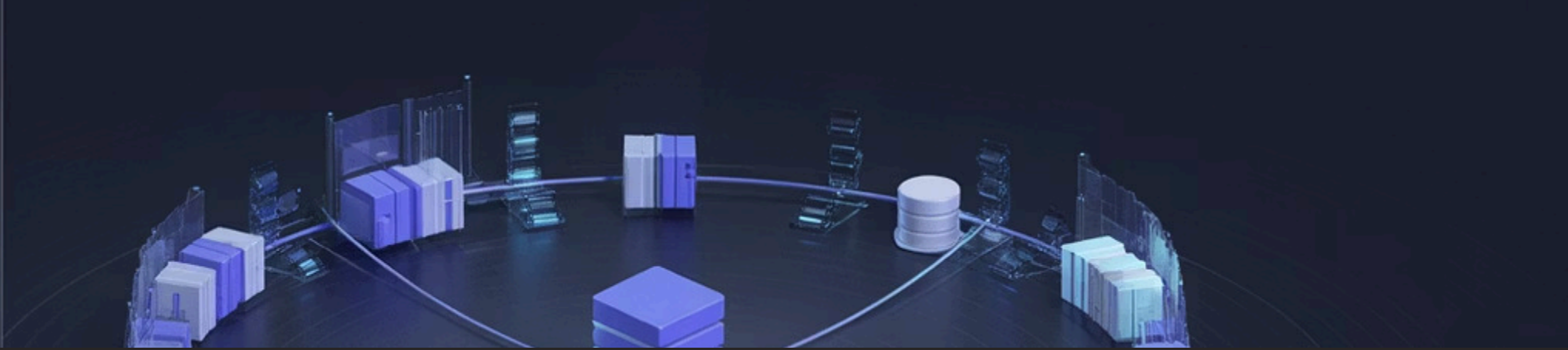
CI/CD Integration

Run tests in CI pipelines

Ensure consistent test environments

Deploy

Output View



Docker Compose for Production

By Ravi Kiran Maroju

Simple Deployments

Suitable for small to medium applications

Single-host deployments

Limitations

No built-in high availability

Limited scaling capabilities

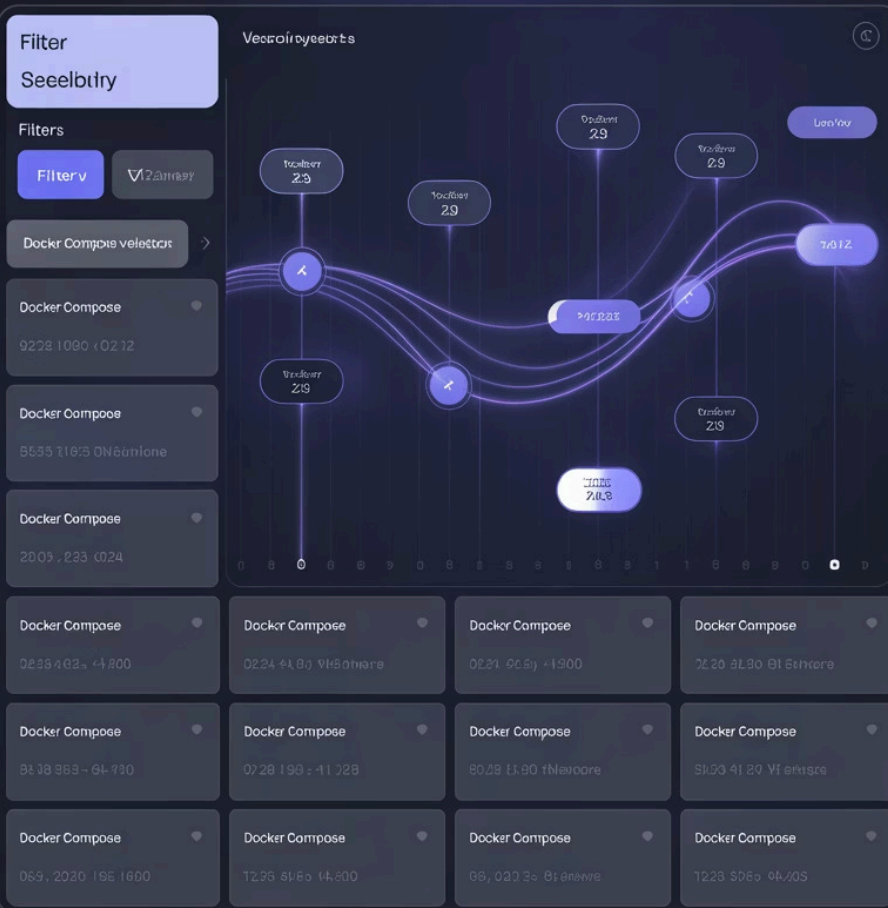
Alternatives

Docker Swarm for multi-host orchestration

Kubernetes for complex deployments

Docker Compose Version Differences

By Ravi Kiran Maroju



Version	Key Features
Version 1	Basic service definitions, links for networking
Version 2	Networks, volumes, dependencies, build arguments
Version 3	Swarm mode support, deploy configuration, secrets
Version 3.8+	Advanced features like GPU support, config templating

Docker Compose vs Docker Swarm

By Ravi Kiran Maroju

Docker Compose

- Single-host deployment
- Simple YAML configuration
- Development-friendly
- Limited scaling

Docker Swarm

- Multi-host deployment
- Built-in orchestration
- High availability
- Load balancing

Docker Compose vs Kubernetes

By Ravi Kiran Maroju

Docker Compose

- Simple configuration
- Quick setup
- Lower learning curve
- Single-host focused

Kubernetes

- Complex configuration
- Enterprise-grade features
- Steep learning curve
- Multi-host, multi-cloud

Docker Compose Best - Practices

Docker Compose Best Practices

By Ravi Kiran Maraju

1 Use version control

Keep your docker-compose.yml in version control

Separate environments

Use override files for different environments

Secure sensitive data

Use .env files and secrets for sensitive information

Pin versions

Use specific image tags instead of 'latest'

More Docker Compose Best Practices

By Ravi Kiran Maroju

1

Organize services logically

Group related services together



Add health checks

Ensure services are truly ready



Configure logging

Set appropriate logging drivers and limits

4

Limit container capabilities

Follow principle of least privilege

Docker Compose Troubleshooting

By Ravi Kiran Maroju



Check logs

`docker-compose logs` to see container output



Inspect containers

`docker-compose exec service bash` to get inside

3

Verify networking

Check if services can reach each other

4

Validate configuration

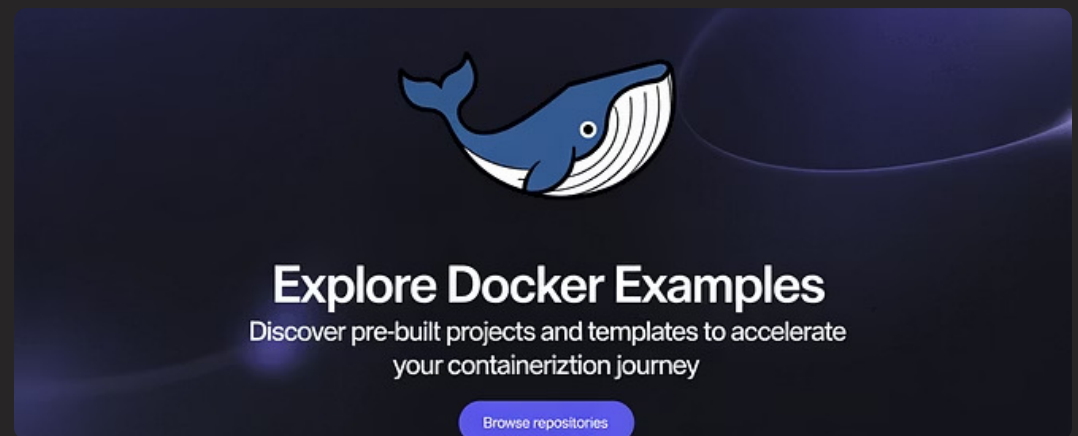
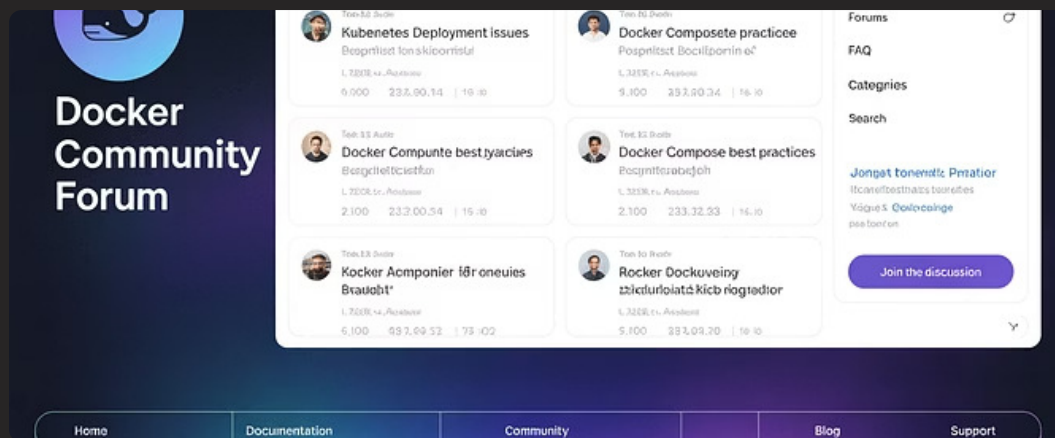
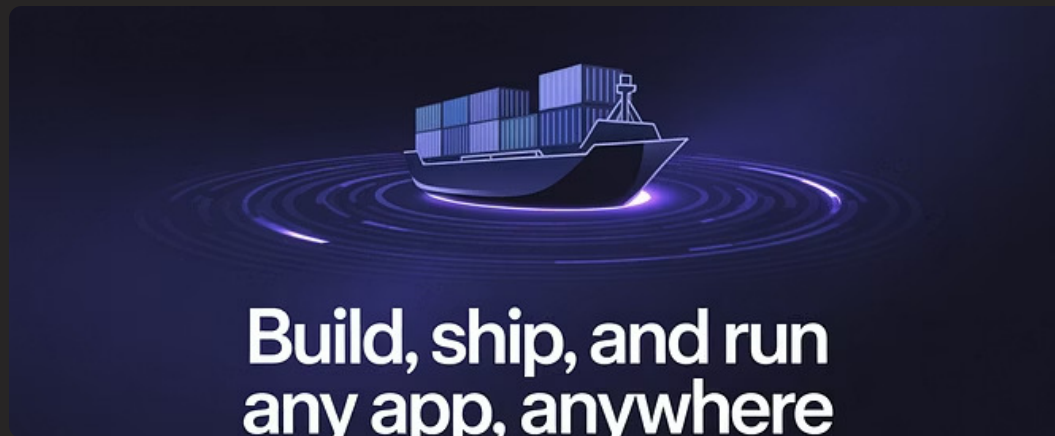
`docker-compose config` to check for errors

Docker Compose
Troubleshooting
and Debugging.



Learning Resources

By Ravi Kiran Maraju



Continue your Docker Compose journey with these valuable resources. The official Docker documentation provides comprehensive guides, while community forums offer practical advice from experienced users. Online courses and tutorials can help you master advanced concepts through hands-on practice.